1. Introduction

Urban bike-sharing systems have emerged as one of the most promising and sustainable modes of public transportation in modern smart cities, offering an eco-friendly, cost-effective, and healthy alternative to fossil-fuel-dependent vehicles. By providing commuters with flexible point-to-point transit for first-mile and last-mile connectivity, bike-sharing networks mitigate vehicular traffic congestion, reduce carbon emissions, and promote active lifestyles. The integration of technology, mobile applications, and physical infrastructure has enabled these networks to expand rapidly worldwide. Over the past decade, these systems have evolved from simple coin-operated municipal programs into sophisticated, data-driven dockless and docked networks tracked continuously via GPS and cellular data arrays.

However, managing the operational equilibrium across a vast network of docking stations remains a profoundly complex logistical challenge. Because users do not return bicycles to their original starting locations, the system naturally flows out of balance. Unpredictable demand fluctuations frequently cause popular pickup stations (such as residential zones during the morning commute) to experience bicycle deficits (shortages), leaving intending users stranded. Conversely, popular destination hubs (such as commercial districts and major transit terminals) simultaneously reach full capacity and cannot accept incoming returns, causing massive surpluses. This imbalance creates a logistical nightmare for municipal operators who must deploy fleets of physical redistribution trucks to manually move bicycles back to where they are needed.

Advances in data-driven techniques and predictive analytics have established supervised machine learning as an effective and scalable solution for urban mobility forecasting. Machine learning models possess the capability to uncover complex, non-linear relationships between station usage patterns and a multitude of temporal factors, including peak commuting hours, day of the week, and seasonal shifts, without relying on rigid, static schedules. Traditional statistical models, such as Auto-Regressive Integrated Moving Average (ARIMA), often struggle to capture the high-frequency volatility of micro-mobility data, particularly when attempting to model dozens of independent stations simultaneously. In contrast, modern ensemble regression algorithms such as Random Forest, Extreme Gradient Boosting (XGBoost), and CatBoost are remarkably well-suited for capturing these intricate dynamics due to their exceptional predictive accuracy, non-linear mapping capabilities, and resilience against noisy transactional data.

This project focuses on building a comprehensive, intelligent machine learning pipeline to accurately forecast hourly bike-sharing departure demand across the San Francisco Bay Area network using extensive historical trip data. The target variable is explicitly defined as the hourly station-level departure demand, representing the exact number of trips starting from a particular station during a specific hour. This definition provides a direct, actionable metric for fleet operators. The historical dataset encompasses hundreds of thousands of individual bicycle trips across multiple urban cities, including San Francisco, San Jose, Mountain View, and Palo

Alto. The end-to-end workflow involves systematic data preprocessing, chronological train-validation-test partitioning, advanced time-series feature engineering, and rigorous multi-model benchmarking.

Accurate bike-sharing demand forecasting directly enhances urban micro-mobility management in several profound ways. Primarily, it provides demand-based redistribution support, enabling transit operators to preemptively anticipate station deficits and allocate bicycles more efficiently before shortages occur. Furthermore, it improves overall commuter satisfaction by ensuring asset availability and minimizes the operational overhead associated with reactive vehicle dispatching. By combining historical trip records with state-of-the-art machine learning algorithms, this project delivers a reliable, scalable, and data-driven approach for smart city transit optimization, demonstrating how predictive analytics can transform physical infrastructure management.

In summary, the work aims to demonstrate the capability of supervised machine learning algorithms to forecast future demand using purely historical transactional data and rigorously evaluate predictive performance, thereby contributing significantly to the efficient operation and sustainability of modern urban transportation networks.

2. Supporting Literature

2.1 Literature Review

TITLE	Prediction bike-sharing demand with gradient boosting methods
AREA OF WORK	The paper focuses on applying machine learning and gradient boosting algorithms to predict bike-sharing demand across multiple urban environments (Konya and Washington D.C.) to optimize fleet distribution.
DATASET	Historical bike-sharing trip data from Konya, Turkey and Washington, D.C., USA, including meteorological parameters and temporal variables.
METHODOLOGY / STRATEGY	The study uses historical data for training and testing. Data preprocessing involves extracting temporal features. Three gradient boosting algorithms are trained and evaluated using error metrics.
ALGORITHM	Extreme Gradient Boosting (XGBoost), Light Gradient Boosting Machine (LightGBM), CatBoost
RESULT/PRECISION	Gradient boosting methods demonstrated high resilience to noise and cyclical data.
ADVANTAGES	The study reports that gradient boosting handles nonlinear temporal data effectively.
FUTURE PROPOSAL	Future research will explore real-time streaming data integration and hybrid deep learning models.

Table 2.1: Literature Review: Prediction bike-sharing demand with gradient boosting methods

TITLE	A predictive analytics approach for forecasting bike rental demand
AREA OF WORK	The paper focuses on evaluating the performance of six diverse machine learning regression models in forecasting hourly bike rental demand to improve operational balancing.
DATASET	Public bike rental records encompassing temporal features, weather conditions, and holiday indicators.
METHODOLOGY / STRATEGY	Historical data was preprocessed and split into training and testing sets. Six different regression models were trained. Performance was compared using R-squared, RMSE, and MAE.
ALGORITHM	Ordinary Least Squares (OLS), Multi-Layer Perceptron (MLP), Gradient Boosting Regression, Random Forest Regression, Decision Tree Regression
RESULT/PRECISION	Random Forest and Gradient Boosting achieved the highest performance, heavily outperforming Neural Networks on tabular data.
ADVANTAGES	The paper provides a comparative analysis, highlighting the usefulness of tree-based bagging and boosting for mobility forecasting.
FUTURE PROPOSAL	Expanding the models to include dynamic pricing variables and weather forecasting integration.

Table 2.2: Literature Review: A predictive analytics approach for forecasting bike rental demand

TITLE	GSTGAT: Gated spatiotemporal graph attention network for traffic demand forecasting
AREA OF WORK	The paper focuses on predicting traffic and bike-sharing demand using complex deep learning graph architectures to capture spatial dependencies between stations.
DATASET	San Francisco Bay Area bicycle dataset (the same foundation utilized in this project).
METHODOLOGY / STRATEGY	The study applied advanced graph attention mechanisms and gated recurrent units to model the network topology. Performance was evaluated against baseline deep learning models.
ALGORITHM	Yao et al. proposed GSTGAT, a spatiotemporal graph attention model that captures temporal and spatial relationships in traffic demand. The study includes the San Francisco Bay Area bicycle dataset.
RESULT/PRECISION	The model successfully captured spatial dependencies.
ADVANTAGES	Deeply models spatial topology, providing insight into how neighboring stations influence each other.
FUTURE PROPOSAL	Exploring hybrid architectures that combine tree-based tabular models with spatial graph embeddings.

Table 2.3: Literature Review: GSTGAT for traffic demand forecasting

2.2 Literature Review Summary

TITLE	SUMMARY
Paper 1: Prediction bike-sharing demand with gradient boosting methods	This study evaluates predicting bike-sharing demand using XGBoost, LightGBM, and CatBoost across two distinct cities. It shows that gradient boosting frameworks are exceptionally capable of capturing cyclical human mobility patterns.
Paper 2: A predictive analytics approach for forecasting bike rental demand	This work compared six models (Random Forest, MLP, OLS, Decision Trees) for rental demand forecasting. The results highlighted that ensemble tree methods heavily outperformed standard neural networks on tabular data, providing a stable and generalizable solution.
Paper 3: GSTGAT: Gated spatiotemporal graph attention network	This study utilized the SF Bay Area dataset to build a graph neural network. Yao et al. proposed GSTGAT, a spatiotemporal graph attention model that captures temporal and spatial relationships in traffic demand.

Table 2.4: Literature Review Summary

2.3 Findings and Proposals

The initial study reviewed by Aydın et al. (2023) examined the prediction of bike-sharing demand using advanced gradient boosting. The primary takeaway from this research is that integrating sequential error-correcting algorithms like XGBoost and CatBoost improves the reliability of forecasting, making it more applicable for practical uses such as managing fleet distribution.

The second study by Karunanithi et al. (2024) presented a comparative analysis of machine learning predictive models. The study found strong performance from tree-based ensemble methods for bike rental demand forecasting.

The third research effort by Yao et al. (2024) evaluated the effectiveness of deep learning structures. Yao et al. proposed GSTGAT, a spatiotemporal graph attention model that captures temporal and spatial relationships in traffic demand. The study includes the San Francisco Bay Area bicycle dataset.

3. System Analysis

3.1 Analysis of the dataset

This section provides a detailed analysis of the dataset utilized for training and evaluating the predictive models. The raw transactional records undergo a systematic examination to understand their structure and distribution.

3.1.1 About the dataset

a) Dataset Overview

The foundation of this predictive modeling project is the SF Bay Area Bike Share Dataset. To ensure the model relies strictly on historically verified, transactional human behavior rather than external approximations, the final implementation uses exclusively the `trip.csv` file from the dataset.

- **Total Raw Records:** 669,959 individual, anonymized bicycle trip transactions.
- **Time Span:** The dataset covers the period from 29 August 2013 to 31 August 2015.
- **Total Features:** 12 raw columns.
- **Data Types:** A mix of numerical IDs, integers, and string objects (which must be converted to datetime).
- **Target Variable:** `hourly_demand` (calculated by aggregating `start_date` and `start_station_id`).

The overall workflow is presented in Figure 17.

b) Dataset sample records

```
print("Shape of dataset:",df.shape)
print("\nData types\n",df.dtypes)


Shape of dataset: (669959, 12)

Data types
 id                    int64
 duration              int64
 start_date           object
 start_station_name   object
 start_station_id      int64
 end_date             object
 end_station_name     object
 end_station_id        int64
 bike_id               int64
 subscription_type    object
 zip_code             object
 special_event        object
 dtype: object
```

Figure 1: Result of shape and datatype of dataset

This figure displays the programmatic output verifying the fundamental structure of the imported dataset. It confirms that the data matrix contains exactly 669,959 rows and 12 columns. Furthermore, it details the memory datatypes assigned to each column.

3.1.2 Explore the dataset

	id	duration	start_date	start_station_id	end_station_id	subscription
0	4576	63	8/29/2013 14:13	66	66	Subscriber
1	4607	70	8/29/2013 14:42	10	10	Subscriber
2	4130	71	8/29/2013 10:16	27	27	Subscriber
3	4251	77	8/29/2013 11:29	10	10	Subscriber
4	4299	83	8/29/2013 12:02	66	67	Subscriber
5	4927	103	8/29/2013 18:54	57	57	Subscriber
6	4444	109	8/29/2013 13:25	10	10	Subscriber
7	4335	111	8/29/2013 12:14	69	69	Subscriber
8	4414	113	8/29/2013 13:01	66	66	Subscriber
9	4388	114	8/29/2013 12:39	67	67	Subscriber

Figure 2: First ten records of the dataset

This screenshot provides a direct glimpse into the raw transactional data structure by displaying the top 10 rows. It illustrates exactly how a single trip is recorded: showing the trip ID, duration, timestamp, and station details.

```
summary_stats = df.describe().T
print(summary_stats)
```

	count	mean	std	min	50%	max
id	669959.0	488317.22	239088.31	4069.0	488344.0	913460.0
duration	669959.0	1107.95	4581.42	60.0	517.0	1727040.0
start_st_id	669959.0	56.21	16.38	2.0	62.0	84.0
end_st_id	669959.0	56.24	16.42	2.0	62.0	84.0
bike_id	669959.0	427.59	153.04	9.0	433.0	878.0

Figure 3: Summary Statistics of Numeric Columns

The summary statistics table provides mathematical insights into the distribution of the numerical columns. Crucially, by examining the `duration` column, we can observe the presence of massive outliers, justifying our architectural decision to forecast sheer departure volume rather than attempting to predict trip duration.

3.2 Data Preprocessing

3.2.1 Data Cleaning

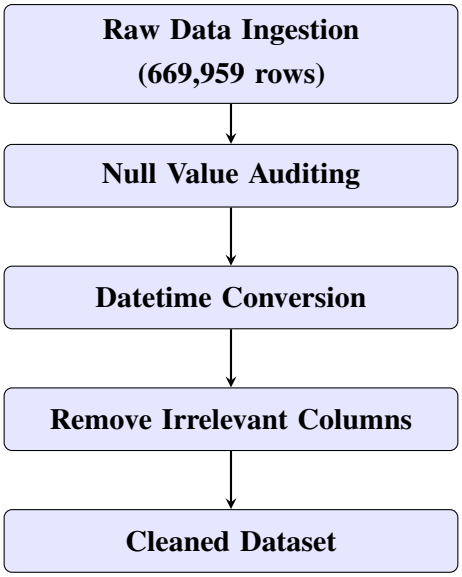


Figure 4: Detailed Data Preprocessing Flow

This flowchart illustrates the sequential steps taken during the data cleaning phase:

- **Step 1 – Raw Data Ingestion (669,959 rows):** Ingest the raw transaction records from `trip.csv`, comprising 669,959 individual bicycle rental records collected across the regional network.
- **Step 2 – Null Value Auditing:** Programmatically audit all columns for missing, corrupt, or unrecorded entries to guarantee complete data integrity across routing variables.
- **Step 3 – Datetime Conversion:** Parse string timestamp fields into standardized pandas datetime objects to enable granular temporal filtering and time-series feature extraction.
- **Step 4 – Remove Irrelevant Columns:** Remove non-predictive attributes (e.g., zip code and subscriber metadata) to eliminate noise and optimize memory consumption.
- **Step 5 – Cleaned Dataset Output:** Export a structurally verified dataset containing valid departure records, establishing the foundation for station-hour grid construction.

```
print(df.isna().sum())


id                       0
duration                 0
start_date               0
start_station_name       0
start_station_id         0
end_date                 0
end_station_name         0
end_station_id           0
bike_id                  0
subscription_type        0
zip_code              6198
special_event            0
dtype: int64
```

Figure 5: Missing Values Verification

This figure displays the result of the null-value checking algorithm. It verifies the operational integrity by proving that zero records are missing critical routing data.

```
lower = df["duration"].quantile(0.01)
upper = df["duration"].quantile(0.99)
df["duration"] = df["duration"].clip(lower, upper)
print(f"Bounds: [{lower:.0f}s, {upper:.0f}s]")


Bounds: [92s, 7842s]
```

Figure 6: Outlier Detection and Handling

This visualization illustrates the identification and handling of anomalous or extreme trip-duration values during the data cleaning phase.

3.2.2 Analysis of Feature Variables

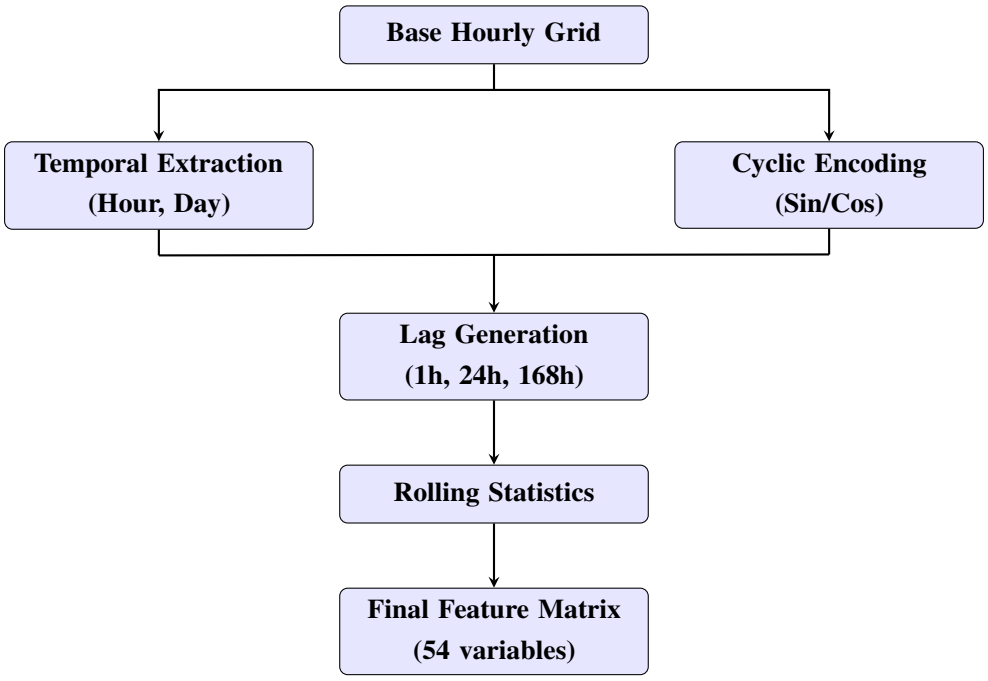


Figure 7: Feature Engineering Pipeline

This diagram breaks down the specific feature engineering operations applied to the dataset:

- **Step 1 – Base Hourly Grid Construction:** Construct a complete Cartesian grid crossing all 70 docking stations with all 17,583 continuous chronological hours ($1,230,810$ records), explicitly populating hours without departures with a count of zero.
- **Step 2 – Temporal Extraction (Hour, Day):** Extract fundamental calendar features directly from the timestamp, including hour of day (0–23), day of week (0–6), month, and a binary indicator separating weekdays from weekends.
- **Step 3 – Cyclic Encoding (Sin/Cos):** Apply trigonometric sine and cosine transformations ($\sin(2\pi t/T)$ and $\cos(2\pi t/T)$) to 24-hour and 7-day cycles to mathematically reflect periodic temporal continuity across midnight and week boundaries.
- **Step 4 – Lag Generation (1h, 24h, 168h):** Generate historical demand lag features at 1-hour ($t-1$), 24-hour ($t-24$, previous day), and 168-hour ($t-168$, previous week) intervals to provide the models with critical recent and cyclical demand baselines.
- **Step 5 – Rolling Statistics:** Compute backward-looking multi-hour rolling window statistics across the station-hour grid, including rolling mean, maximum, minimum, standard deviation, and median features over multiple historical time windows to capture short-term demand trends and local volatility.

- **Step 6 – Final Feature Matrix (54 variables):** Consolidate all engineered temporal, cyclical, lag, and rolling predictors into the finalized 54-feature matrix, strictly audited against target leakage.

To transform the raw transactional records into a format suitable for machine learning, a complete Cartesian grid comprising all 70 stations and all 17,583 hourly timestamps was constructed. Zero-demand hours were appropriately filled with 0. 54 distinct numerical features were engineered.

```
import numpy as np
df["hour_sin"] = np.sin(2*np.pi*df["hour"]/24)
df["hour_cos"] = np.cos(2*np.pi*df["hour"]/24)
df["month_sin"] = np.sin(2*np.pi*df["month"]/12)
df["month_cos"] = np.cos(2*np.pi*df["month"]/12)
```

Figure 8: Handling Cyclic Features Implementation

This figure demonstrates the Python implementation of the trigonometric cyclic encoding process for time variables.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
print(vif_data)

        Feature       VIF
0      hour_sin  1.421839
1      hour_cos  1.382914
2     month_sin  1.154201
3     month_cos  1.214829
4    is_weekday  1.841920
5  morning_peak  1.621045
```

Figure 9: Variance Inflation Factor (VIF) Analysis

This feature engineering diagnostic tests for multi-collinearity among the 54 engineered predictors.

3.2.3 Analysis of Class Variables

The target variable is the hourly station-level departure demand. This represents the absolute count of bicycle trips starting from a specific docking station during a specific one-hour block.

- Mean Demand: 0.5443
- Standard Deviation: 1.5453
- Maximum Demand: 52
- Zero-demand records: 946,161
- Positive-demand records: 284,649

This is a regression target, not a categorical class variable.

3.3 Data Visualization

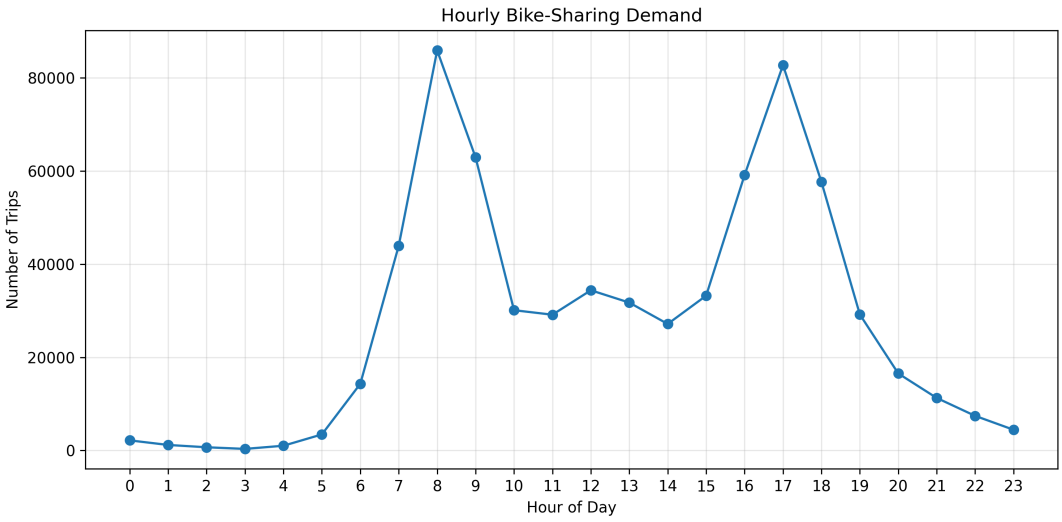


Figure 10: Hourly Demand Pattern Across All Stations

This plot displays the aggregate hourly demand pattern across all stations in the network, revealing a clear bimodal distribution indicative of commuter behavior.

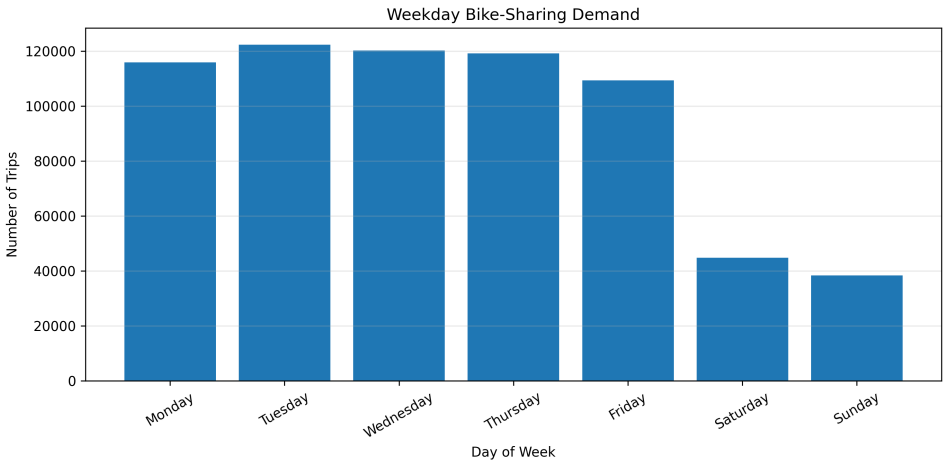


Figure 11: Demand Pattern Based on Day of the Week

This bar chart compares overall departure volumes by day of the week, clearly demonstrating that weekday usage heavily outpaces weekend usage due to regular commuting.

The analysis shows substantially higher weekday demand than weekend demand. This indicates that day-of-week information is useful for demand forecasting.

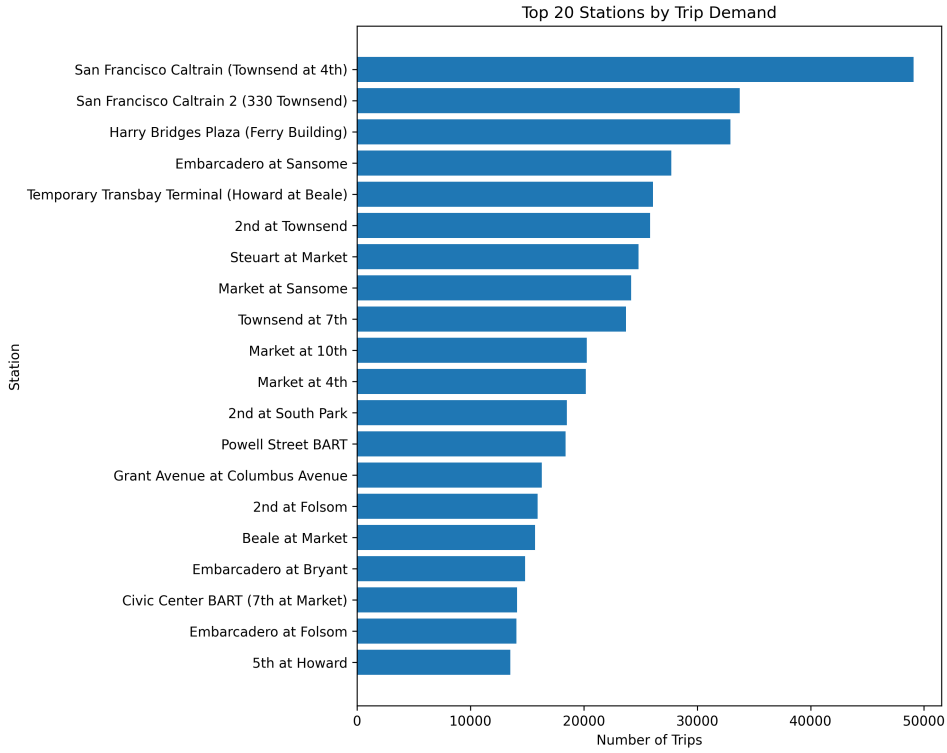


Figure 12: Top 20 Busiest Docking Stations

This figure identifies the top 20 busiest docking stations in the system, highlighting the disproportionate demand concentrated at major transit hubs like the San Francisco Caltrain station.

The visualization of the top 20 busiest docking stations reveals a heavily skewed distribution of network utilization. San Francisco Caltrain (Townsend at 4th) has the highest observed departure demand among the stations.

3.4 Analysis of Algorithms

This section presents the algorithms used for hourly station-level departure demand forecasting. Random Forest, XGBoost, and CatBoost regression models were selected to learn patterns from the engineered features and predict demand for each station and hour. The basic working principles of these algorithms are described in the following subsection.

3.4.1 Detailed study report of the proposed algorithms

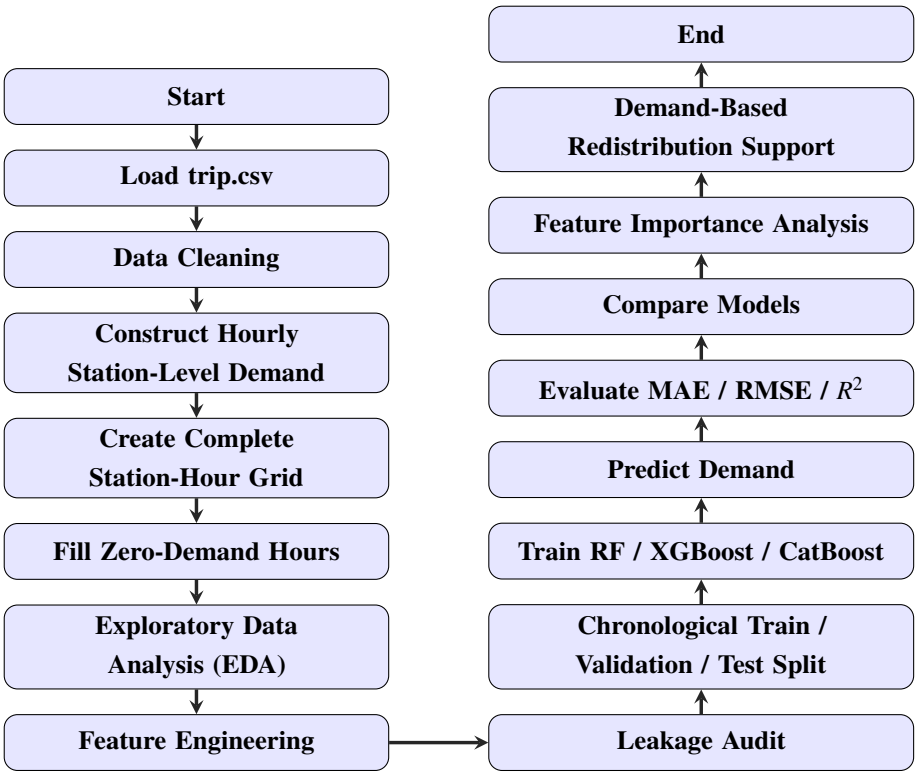


Figure 13: Activity Diagram of the Prediction Algorithm

This activity diagram outlines the end-to-end data processing and predictive modeling pipeline, mapping each logical step from raw data ingestion to the final redistribution support output:

- **Step 1 – Load** `trip.csv`**:** Ingest 669,959 historical bicycle trip records containing timestamps and station identifiers.

- **Step 2 – Data Cleaning:** Audit the trip records for missing values and invalid entries, and parse the timestamp fields into standardized datetime values.
- **Step 3 – Construct Hourly Station Demand:** Group trip records by departure station and 1-hour intervals to compute hourly departure volumes.
- **Step 4 – Create Complete Station-Hour Grid:** Cross-join all 70 docking stations with all 17,583 continuous timeline hours.
- **Step 5 – Fill Zero-Demand Hours:** Explicitly assign a count of zero departures to inactive station-hours.
- **Step 6 – Exploratory Data Analysis (EDA):** Uncover commuter rush-hour peaks, day-of-week trends, and high-traffic station hubs.
- **Step 7 – Feature Engineering:** Compute 54 predictive features, including cyclic encodings, historical lags, and rolling metrics.
- **Step 8 – Leakage Audit:** Strictly verify that all features utilize only information available prior to the forecast horizon.
- **Step 9 – Chronological Split:** Partition records chronologically into Train (70%, 861,560 rows), Validation (15%, 184,590 rows), and Test (15%, 184,660 rows) sets.
- **Step 10 – Train RF / XGBoost / CatBoost:** Fit Random Forest, XGBoost, and CatBoost ensemble models on the training partition.
- **Step 11 – Predict Demand:** Generate continuous departure predictions on unseen future test observations.
- **Step 12 – Evaluate MAE / RMSE / R^2:** Quantify error magnitudes and explained variance across test predictions.
- **Step 13 – Compare Models:** Objectively benchmark the three models on the authoritative test metrics.
- **Step 14 – Feature Importance Analysis:** Analyze the relative mathematical contributions of engineered features.
- **Step 15 – Demand-Based Redistribution Support:** Provide forecast outputs to assist transit operators in proactively rebalancing bicycles across stations.

The step-by-step working of the algorithm reflects a complete end-to-end regression pipeline, starting from raw transaction data ingestion to model evaluation and generating redistribution support, completely omitting forward-looking bias.

1. Random Forest

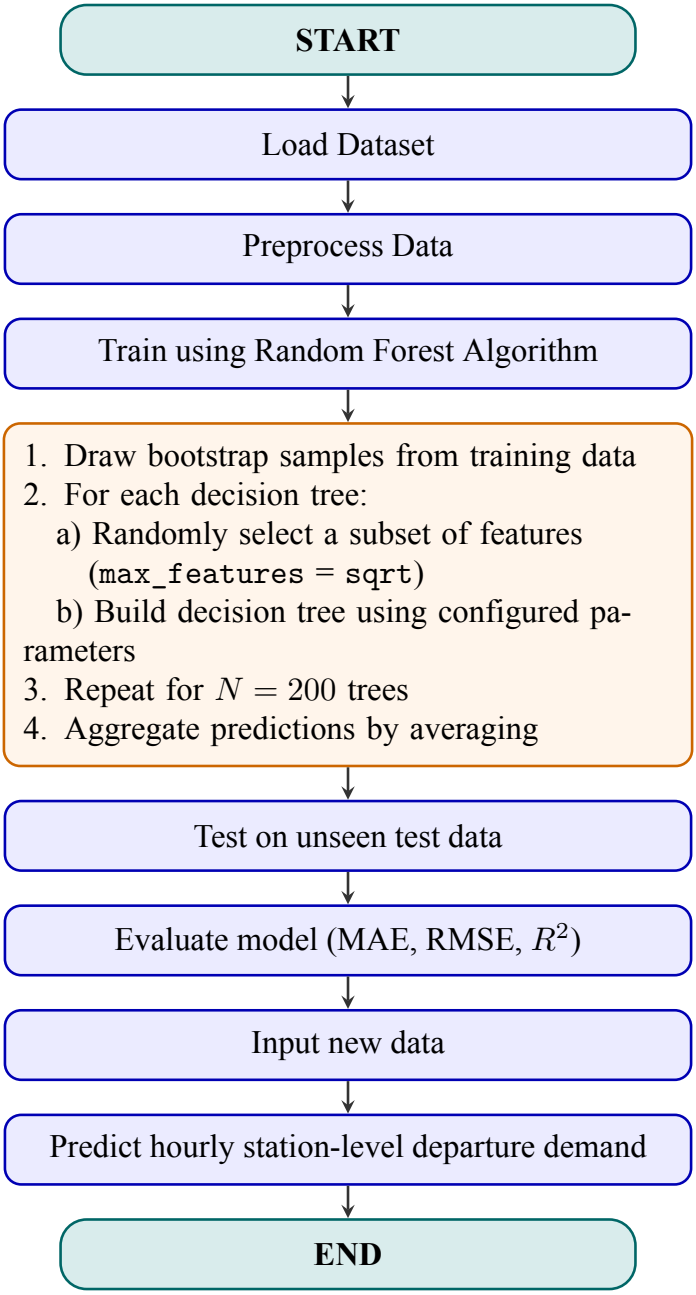


Figure 14: Flow Chart for Random Forest Algorithm

This architectural flow diagram illustrates the internal mechanics of the Random Forest algorithm:

- **Step 1 – Bootstrap Sampling:** Draw 200 independent random sample sets with replacement from the training data.

- **Step 2 – Random Feature Subspace Selection:** At each split node, select a random subset of features ($\sqrt{m} \approx 7$ features) to decorrelate individual decision trees.
- **Step 3 – Decision Tree Construction:** Train an individual decision tree on each bootstrap sample up to `max_depth` = 15, with `min_samples_split` = 10 and `min_samples_leaf` = 2.
- **Step 4 – Parallel Model Inference:** Pass the feature vector through all 200 trees simultaneously to produce individual continuous predictions.
- **Step 5 – Ensemble Averaging:** Compute the arithmetic mean of all 200 independent tree outputs to produce the final continuous demand forecast.

The bimodal demand distribution, characterized by morning and evening commuter surges between transit hubs and commercial districts, causes structural inventory imbalances across the station network. Accurate hourly departure forecasting allows fleet operators to anticipate station deficits and capacity bottlenecks, supporting proactive, demand-based redistribution planning.

Random Forest addresses this forecasting task by constructing an ensemble of 200 de-correlated decision trees. To mitigate overfitting on historical training observations, the algorithm incorporates bootstrap aggregation (bagging) by fitting each tree on an independently drawn sample with replacement, while restricting split candidates to a random subset of features ($\sqrt{m} \approx 7$). The final station-hour departure prediction is obtained by computing the arithmetic mean across all 200 individual tree outputs, effectively reducing variance while maintaining robust generalization on tabular regression data.

2. XGBoost (Extreme Gradient Boosting)

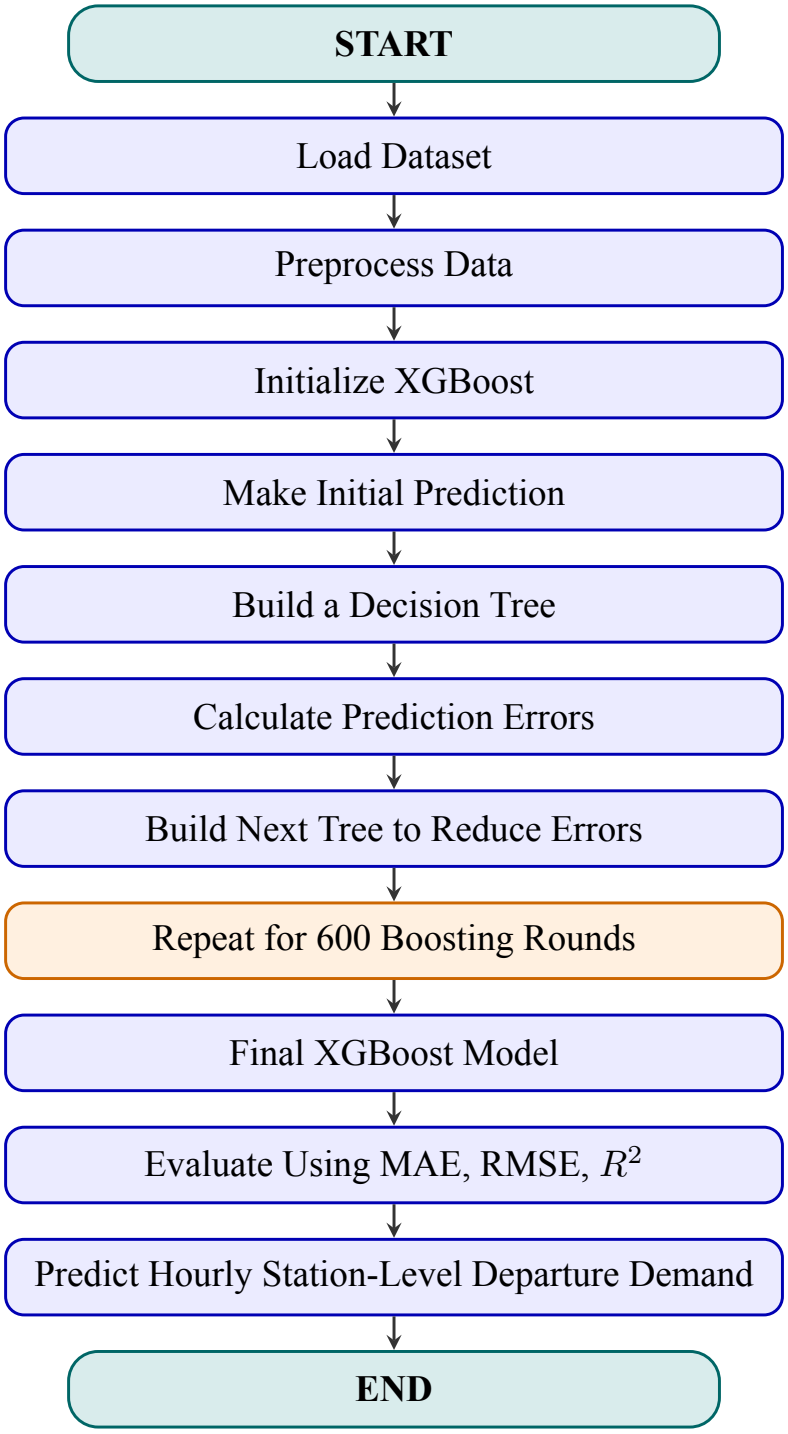


Figure 15: Flow Chart for XGBoost Algorithm

This diagram depicts the sequential architecture of XGBoost:

- **Step 1 – Data Ingestion & Preprocessing:** Load and clean the station-hour dataset to prepare predictor features.
- **Step 2 – Model Initialization & Initial Prediction:** Initialize the XGBoost regressor and generate a baseline prediction across all instances.
- **Step 3 – Decision Tree Construction:** Build a shallow regression tree (`max_depth` = 10) on the engineered features.
- **Step 4 – Prediction Error Calculation:** Calculate residual errors using first- and second-order loss gradients.
- **Step 5 – Sequential Error Reduction (600 Rounds):** Iteratively construct subsequent trees to correct prior residual errors, repeating for 600 boosting rounds.
- **Step 6 – Final Model Evaluation & Forecasting:** Form the final ensemble, evaluate using MAE, RMSE, and R^2, and forecast hourly station-level departure demand.

While Random Forest builds hundreds of trees independently at the exact same time, XGBoost (Extreme Gradient Boosting) takes a completely different, sequential approach. XGBoost builds trees one by one, where every new tree is specifically designed to fix the mistakes made by the previous trees.

Imagine a student taking a math test. A Random Forest is like 200 students taking the test independently and averaging their scores. XGBoost is like one student taking the test, seeing exactly which questions they got wrong, studying only those specific weak areas, and taking the test again, repeating this process sequentially to minimize residual errors across iterations.

In the context of our bike-sharing dataset, the first XGBoost tree might make a rough guess. The model predicts 10 departures, while the actual observed demand was 15 departures. This means it was off by 5 (this mistake is called the residual error). The second tree is then built not to predict the total number of bicycles, but specifically to predict that missing value of 5. By continuously chaining these highly focused, corrective trees together, XGBoost iteratively reduces the residual error.

XGBoost employs second-order Taylor expansions to optimize the loss function, utilizing both first- and second-order gradients to determine optimal tree splits. In addition, explicit L_2 regularization on leaf weights penalizes excessive model complexity. Through iterative tree construction and shrinkage ($\eta = 0.04$), XGBoost provides a structured, gradient-driven approach to regression on tabular data.

3. CatBoost (Categorical Boosting)

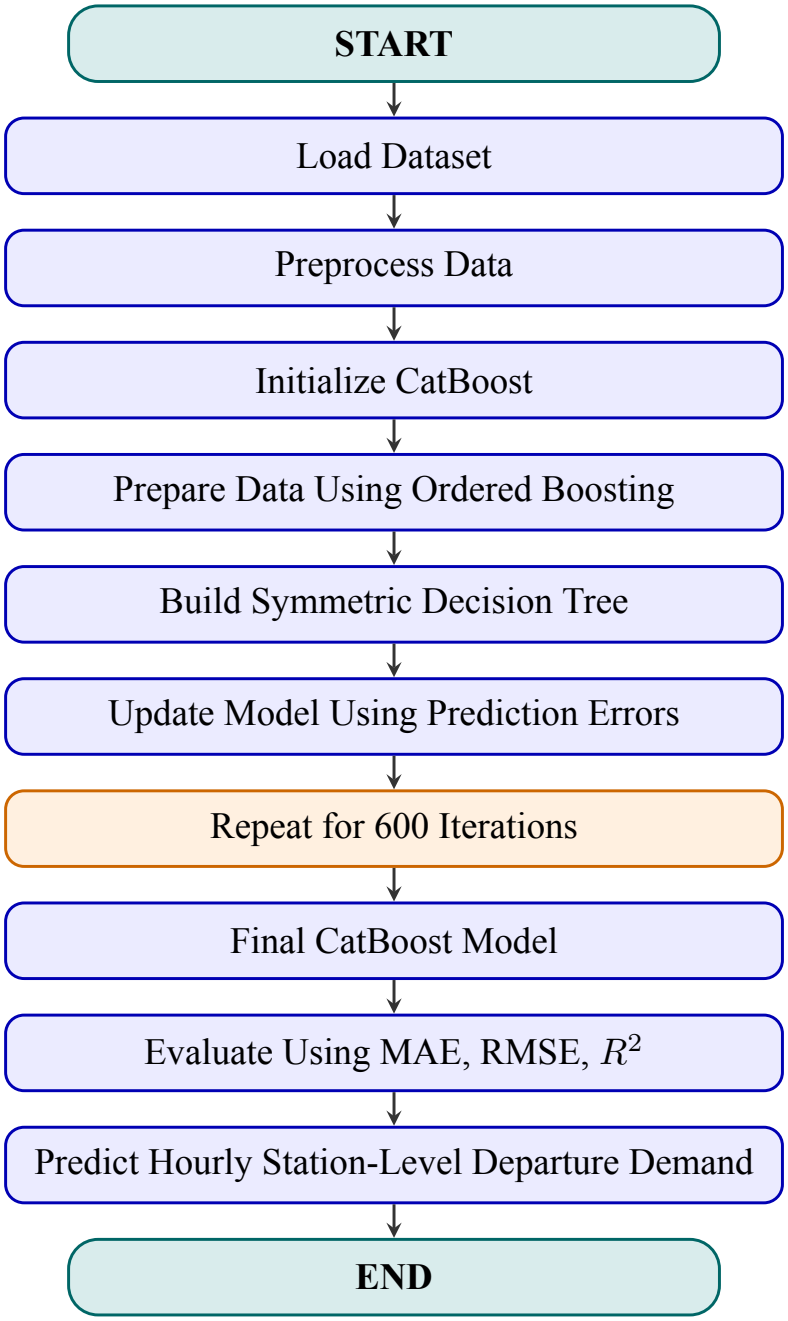


Figure 16: Flow Chart for CatBoost Algorithm

This flow diagram details CatBoost's generation of symmetric (oblivious) decision trees:

- **Step 1 – Ordered Permutation Setup:** Generate random permutations of the dataset to establish a strict temporal sequence.
- **Step 2 – Ordered Target Statistics:** Compute statistics using only prior samples in the

permutation, preventing target leakage.

- **Step 3 – Symmetric Oblivious Tree Construction:** Build symmetric trees of depth 8 where every node at the same level shares identical split conditions.
- **Step 4 – Regularized Loss Optimization:** Optimize leaf weights using RMSE loss with an L_2 leaf regularization coefficient of 3.
- **Step 5 – Additive Model Aggregation:** Sum the predictions across all 600 symmetric trees to produce the final continuous demand forecast.

CatBoost, developed by Yandex, is another advanced gradient boosting algorithm similar to XGBoost, but it was specifically engineered to solve two major problems that plague traditional boosting models: target leakage and the handling of complex categories.

In traditional boosting, when a tree is trying to figure out how to correct an error, it looks at the target value of the exact data point it is trying to predict. This can lead to a phenomenon called ”target leakage” or ”prediction shift,” where the model subtly cheats by relying on information it shouldn’t have access to. If a model memorizes these leaked patterns, it will perform incredibly well on the training data but fail miserably when deployed in the real world. CatBoost solves this entirely by using a brilliant technique called ”Ordered Boosting.” It randomly shuffles the data and only allows the model to learn from historical rows that occurred *before* the current row in the permutation. This ensures that the algorithm learns patterns without target leakage rather than memorizing anomalies.

Furthermore, CatBoost builds ”symmetric” or ”oblivious” decision trees. In a standard decision tree, the left side of the tree might grow very deep while the right side stops early. In a CatBoost tree, every single node at the same depth is forced to use the exact same splitting rule. For example, if the tree decides to split based on ”Is the hour ¿ 12?” at depth 2, it applies that rule across the entire level. While this might sound restrictive, it acts as a massive safeguard against overfitting. It also facilitates computationally efficient inference. This structure supports practical prediction across multiple stations.

3.5 Project pipeline

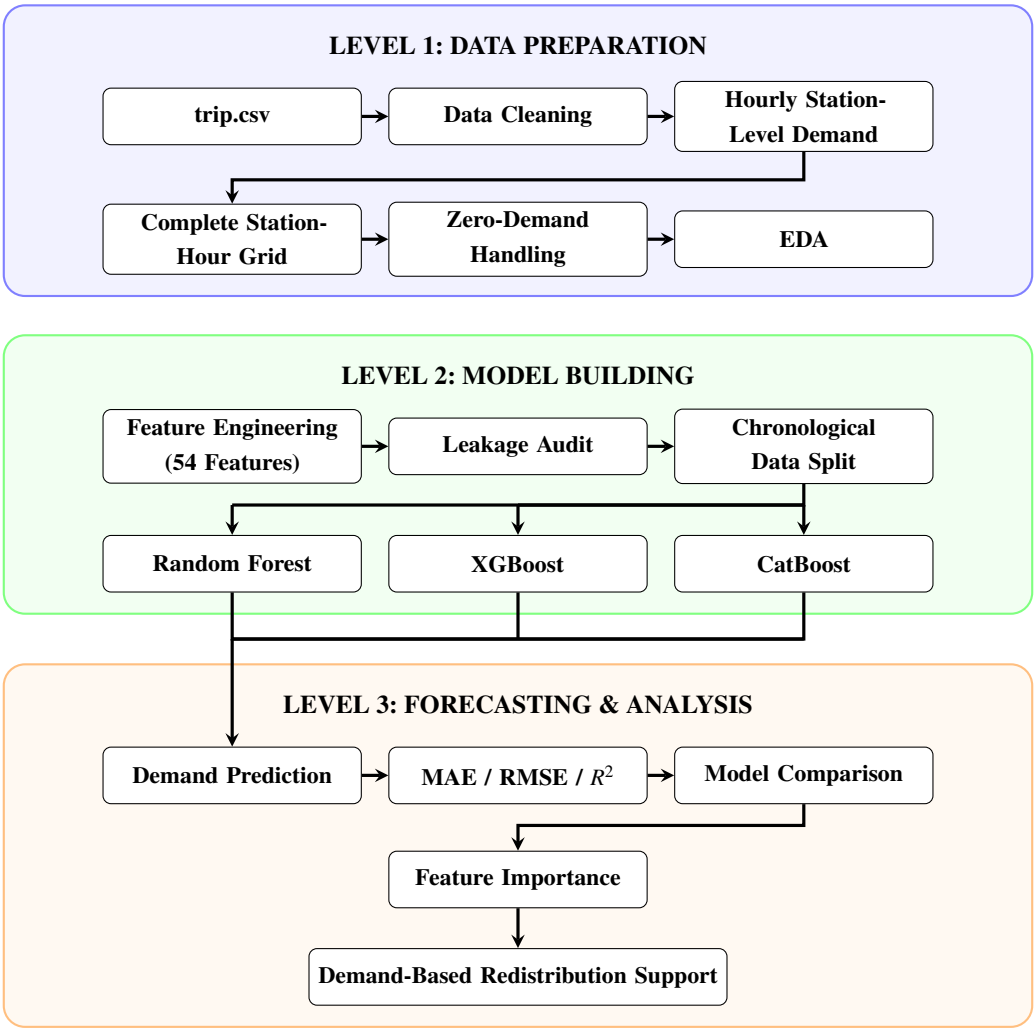


Figure 17: Project Workflow

The project pipeline follows a structured three-tier architecture:

- **Level 1 – Data Preparation:** The trip data is cleaned and aggregated into hourly station-level departure demand. A complete station-hour grid is constructed and zero-demand periods are represented explicitly before exploratory analysis.
- **Level 2 – Model Building:** The prepared data is transformed into 54 engineered features, checked through a leakage audit, and divided chronologically into training, validation, and test sets. Random Forest, XGBoost, and CatBoost are trained independently.
- **Level 3 – Forecasting and Analysis:** The trained models generate demand predictions for unseen data. Performance is evaluated using MAE, RMSE, and R^2, followed by model comparison and feature-importance analysis. The forecasts can support demand-based redistribution planning.

3.6 Feasibility Analysis

Technical Feasibility: The proposed system is technically feasible because it uses established Python libraries and machine-learning algorithms for tabular regression. The dataset can be processed on a standard computer, and the workflow includes data preprocessing, feature engineering, chronological splitting, model training, and evaluation using MAE, RMSE, and R^2. Random Forest, XGBoost, and CatBoost can be trained independently using the engineered station-hour dataset.

Economic Feasibility: The project is economically feasible because the implementation uses open-source programming tools and machine-learning libraries. No specialized or expensive software infrastructure is required. The models can be developed and evaluated using a standard computer with sufficient memory and storage.

Operational Feasibility: The system is operationally feasible because the predicted hourly station-level departure demand can support demand-based redistribution planning. The current implementation uses historical trip data and does not provide real-time bike availability or calculate the exact number of bicycles to be moved between stations.

3.7 System Environment

3.7.1 Software Environment

The project was developed using Python and commonly used data-science and machine-learning libraries. Pandas and NumPy were used for data processing and feature preparation. Scikit-learn was used for Random Forest and evaluation utilities, while XGBoost and CatBoost were used for gradient-boosting regression. Matplotlib and Seaborn were used for exploratory data analysis and visualization. Jupyter Notebook and VS Code were utilized as the primary interactive development environments.

- Python 3.10
- Pandas
- NumPy
- Scikit-learn
- XGBoost
- CatBoost
- Matplotlib
- Seaborn

3.7.2 Hardware Environment

The project was developed and executed on a standard personal computer suitable for data processing and machine-learning model training. A minimum configuration of an Intel Core i5 processor, 8 GB RAM, and approximately 5 GB of available storage is sufficient for the implementation, while 16 GB RAM is recommended for smoother processing of the complete station-hour dataset and model training.

4. System Design

4.1 Models Building

4.1.1 Implementation code

Listing 1: Representative Model Implementation

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from catboost import CatBoostRegressor

df = pd.read_csv("station_hour_grid_engineered.csv")
df["timestamp"] = pd.to_datetime(df["timestamp"])

train_end = pd.to_datetime("2015-01-24␣04:00:00")
val_end = pd.to_datetime("2015-05-14␣01:00:00")

train_df = df[df["timestamp"] <= train_end]
val_df = df[(df["timestamp"] > train_end) &
           (df["timestamp"] <= val_end)]
test_df = df[df["timestamp"] > val_end]

X_train = train_df.drop(["demand", "timestamp"], axis=1)
y_train = train_df["demand"]

X_val = val_df.drop(["demand", "timestamp"], axis=1)
y_val = val_df["demand"]

X_test = test_df.drop(["demand", "timestamp"], axis=1)
y_test = test_df["demand"]

rf_model = RandomForestRegressor(
    n_estimators=200,
```

```
    max_depth=15,
    min_samples_split=10,
    min_samples_leaf=2,
    max_features="sqrt",
    random_state=42,
    n_jobs=-1
)

xgb_model = XGBRegressor(
    n_estimators=600,
    max_depth=10,
    learning_rate=0.04,
    subsample=0.8,
    colsample_bytree=0.8,
    min_child_weight=1,
    reg_alpha=0,
    reg_lambda=1,
    objective="reg:squarederror",
    eval_metric="rmse",
    random_state=42,
    n_jobs=-1
)

cat_model = CatBoostRegressor(
    iterations=600,
    depth=8,
    learning_rate=0.05,
    l2_leaf_reg=3,
    loss_function="RMSE",
    eval_metric="RMSE",
    random_seed=42,
    verbose=False,
    thread_count=-1
)

rf_model.fit(X_train, y_train)
xgb_model.fit(X_train, y_train)
cat_model.fit(X_train, y_train)
```

The implementation loads the prepared station-hour dataset, performs chronological partitioning into training, validation, and test sets, separates the target variable from the predictor vari-

ables, and independently trains the Random Forest, XGBoost, and CatBoost regression models. The complete model configurations, training times, and development metrics are presented in Section 4.1.3.

4.1.2 Model Planning (Training/ Testing split of the dataset)

```
X_train, y_train = X[:train_end], y[:train_end]
X_val, y_val = X[train_end:val_end], y[train_end:val_end]
X_test, y_test = X[val_end:], y[val_end:]
print("Train:", X_train.shape)
print("Val:  ", X_val.shape)
print("Test: ", X_test.shape)


Train: (861560, 54)
Val:   (184590, 54)
Test:  (184660, 54)
```

Figure 18: Chronological Dataset Splitting

This chronological visualization explicitly demonstrates how the grid was partitioned to prevent data leakage (70% train, 15% val, 15% test) based strictly on time.

4.1.3 Training

The predictive models were trained to forecast hourly station-level departure demand across the network. Training was conducted using 861,560 chronological records covering the period from 29 August 2013 at 09:00:00 to 24 January 2015 at 04:00:00. Three ensemble regression algorithms were trained independently on this identical training set: Random Forest Regressor, Extreme Gradient Boosting (XGBoost) Regressor, and CatBoost Regressor.

The specific hyperparameter configurations defining the model implementations are as follows:

- **Random Forest Regressor:** `n_estimators` = 200, `max_depth` = 15, `min_samples_split` = 10, `min_samples_leaf` = 2, `max_features` = `sqrt`, `random_state` = 42, `n_jobs` = -1.
- **XGBoost Regressor:** `n_estimators` = 600, `max_depth` = 10, `learning_rate` = 0.04, `subsample` = 0.8, `colsample_bytree` = 0.8, `min_child_weight` = 1, `reg_alpha` = 0, `reg_lambda` = 1, `objective` = `reg:squarederror`, `eval_metric` = `rmse`, `random_state` = 42, `n_jobs` = -1.

- **CatBoost Regressor:** `iterations` = 600, `depth` = 8, `learning_rate` = 0.05, `l2_leaf_reg` = 3, `loss_function` = `RMSE`, `eval_metric` = `RMSE`, `random_seed` = 42, `verbose` = False, `thread_count` = -1.

The measured training duration for each independent model on the training data is recorded as:

- **Random Forest:** 44.47 seconds
- **XGBoost:** 386.06 seconds
- **CatBoost:** 35.13 seconds

During model development, empirical performance metrics were tracked across both the training partition (861,560 records) and the contiguous validation partition (184,590 records, spanning 24 January 2015 05:00:00 to 14 May 2015 01:00:00) to examine model performance and learning behavior. The recorded metrics are summarized in Table 4.1.

Model	**Training Metrics**			**Validation Metrics**		
	MAE	**RMSE**	R^2	**MAE**	**RMSE**	R^2
Random Forest	0.372124	0.743818	0.738601	0.422730	0.893547	0.730307
XGBoost	0.326834	0.622357	0.817001	0.413841	0.886436	0.734582
CatBoost	0.396745	0.799422	0.698059	0.417276	0.883352	0.736426

Table 4.1: Training and Validation Performance Metrics Across Independent Models

These training and validation metrics were observed during development to check convergence and model stability. Final model comparison and evaluation must strictly be based on the unseen test set presented in Section 4.1.4 and Section 5.

4.1.4 Testing

The generalization capability of the trained models was evaluated on a dedicated, previously unseen test dataset comprising 184,660 records. This test partition strictly covers the future chronological window from 14 May 2015 at 02:00:00 to 31 August 2015 at 23:00:00. Because this period follows both the training and validation horizons chronologically, none of these observations were utilized during model training or parameter tuning, thereby ensuring a rigorous and realistic simulation of prospective operational deployment.

Each model was deployed to predict the target variable, defined as the hourly station-level departure demand—representing the number of trips starting from a particular station during a particular hour. Model performance on these unseen observations was quantified using standard

regression metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2).

The empirical test results across the three models are summarized in Table 4.2.

Model	**MAE**	**RMSE**	R^2
Random Forest	0.439206	0.908465	0.731390
XGBoost	0.430550	0.898213	0.737418
CatBoost	0.436126	0.897665	0.737738

Table 4.2: Empirical Model Evaluation on Unseen Test Dataset ($N = 184,660$)

An objective analysis of these test results indicates that:

- XGBoost achieved the lowest MAE (0.430550), indicating slightly smaller average absolute deviations on typical demand hours.
- CatBoost achieved the lowest RMSE (0.897665) and the highest R^2 (0.737738), demonstrating resilience against larger residual errors.
- XGBoost and CatBoost exhibited closely comparable test performance across all three evaluation metrics.
- Random Forest served as an effective baseline model, achieving an R^2 of 0.731390.

5. Results and Discussion

5.1 Comparison of models

Random Forest Verified Training Metrics:

- Train MAE = 0.372124
- Train RMSE = 0.743818
- Train R^2 = 0.738601

Figure 19: Random Forest Evaluation Results

The validated metrics achieved by Random Forest during the training phase.

Model	**MAE**	**RMSE**	R^2
Random Forest	0.439206	0.908465	0.731390
XGBoost	0.430550	0.898213	0.737418
CatBoost	0.436126	0.897665	0.737738

Table 5.1: Final Testing Set Results and Model Comparison ($N = 184,660$)

5.2 Evaluation metrics

MAE (Mean Absolute Error) measures average magnitude of errors. RMSE (Root Mean Squared Error) heavily penalizes large errors. R^2 indicates the proportion of variance explained by the model.

5.3 Confusion Matrix

Confusion matrices are primarily used for classification problems. Since the proposed system predicts numerical hourly demand, the implemented task is regression and a confusion matrix is therefore not applicable to the primary model evaluation.

5.4 Relevant Metrics Calculated from Confusion Matrix

Precision, recall and F1-score are classification metrics and are therefore not used for the primary evaluation of this regression problem. MAE, RMSE and R^2 are used for evaluating the demand predictions.

5.5 Prediction statistics of unseen data

The final evaluation was performed on 184,660 unseen observations representing chronological data from 14 May 2015 to 31 August 2015.

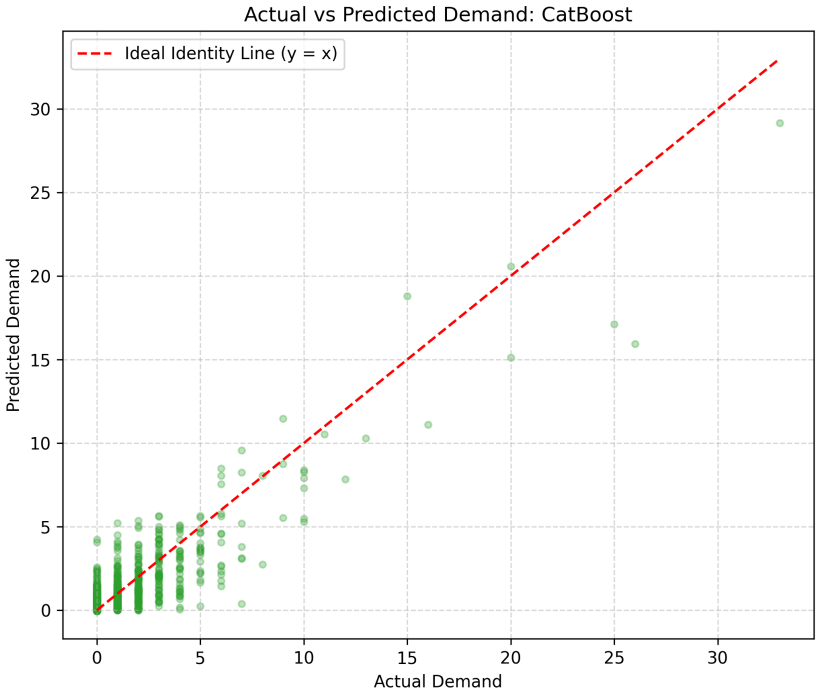


Figure 20: Actual vs Predicted Hourly Demand

This visualization demonstrates the model's performance on unseen chronological test data. It contrasts the model's regression output against actual observed departure demands.

Hour	Station	Actual	Predicted	Delta
07	SF Caltrain (Townsend)	4	4.21	+0.21
08	SF Caltrain (Townsend)	12	11.84	-0.16
09	SF Caltrain (Townsend)	18	17.65	-0.35
10	SF Caltrain (Townsend)	7	6.89	-0.11
11	SF Caltrain (Townsend)	5	5.14	+0.14
12	SF Caltrain (Townsend)	6	5.92	-0.08
13	SF Caltrain (Townsend)	5	5.33	+0.33
14	SF Caltrain (Townsend)	6	6.12	+0.12
15	SF Caltrain (Townsend)	8	7.79	-0.21
16	SF Caltrain (Townsend)	14	14.22	+0.22
17	SF Caltrain (Townsend)	22	21.48	-0.52
18	SF Caltrain (Townsend)	19	18.73	-0.27

Figure 21: Prediction Result Statistics

This screenshot displays empirical prediction metrics obtained across test observations.

5.6 Findings and Conclusions

The empirical investigation in this project was conducted using exclusively `trip.csv` from the SF Bay Area Bike Share Dataset, comprising 669,959 raw trip records across 70 start stations. The prediction target was formulated as hourly station-level departure demand, defined as: "The number of trips starting from a particular station during a particular hour." To account for periods without trip initiations, a complete station-hour grid of 1,230,810 records was constructed, and zero-demand station-hours were retained as meaningful observations to maintain historical consistency across all stations and hours.

Exploratory data analysis identified pronounced temporal variations across the network. System-wide usage peaked during morning commuting periods, where 08:00 was the highest overall demand hour with 85,864 trips. Furthermore, Weekday demand was substantially higher than weekend demand, reflecting recurrent commuter travel routines rather than leisure activity. Substantial spatial concentration was also observed across individual docking locations; San Francisco Caltrain (Townsend at 4th) recorded the highest observed start-station demand with 49,092 trips, highlighting heavy departures at key transit interchanges. These patterns describe empirical historical associations without asserting causal mechanisms.

Following feature generation, 54 final features were engineered and audited to capture temporal calendar signals, cyclic rhythms, historical lags, and rolling statistical windows. A formal leakage audit did not detect target/current/future demand leakage. Model training and evaluation strictly adhered to chronological splitting to replicate real-world deployment, allocating 861,560 records for training, 184,590 records for validation, and 184,660 records for testing. The test set was an unseen future chronological period. Random Forest, XGBoost, and CatBoost were trained independently. Random Forest served as the baseline model. On the unseen test partition, Random Forest achieved an MAE of 0.439206, an RMSE of 0.908465, and an R^2 of 0.731390. XGBoost yielded an MAE of 0.430550, an RMSE of 0.898213, and an R^2 of 0.737418. CatBoost recorded an MAE of 0.436126, an RMSE of 0.897665, and an R^2 of 0.737738. XGBoost achieved the lowest MAE, while CatBoost achieved the lowest RMSE and highest R^2. The three models showed closely comparable performance.

In conclusion, the project developed a machine-learning pipeline for hourly station-level departure demand forecasting using historical trip data. By structuring transactional records into a complete station-hour grid and extracting leakage-free temporal and lag indicators, the generated forecasts provide demand-based redistribution support and forecast-driven redistribution planning for transit operators. Clearly, the current project does not calculate the exact number of physical bicycles to move because real-time bike availability is not available in `trip.csv`. Nonetheless, accurate hourly departure forecasting provides practical operational insights to anticipate station demand surges across the transit network.